TESTES DE SOFTWARE · MACHINE LEARNING

Fuzz Testing em Machine Learning

ResNet-50, ViT-Base e o tokenizer do DistilBERT testados com fuzzing guiado por cobertura

⚠ **Conteúdo gerado por IA (Claude):** código, testes, textos e slides. Revise antes de usar ou apresentar.

28 de outubro de 2026 · leonardorh18/ml-fuzzy-tests

O QUE É FUZZ TESTING

Gerar milhares de entradas até alguma quebrar o sistema

O fuzzer cria entradas novas, aleatórias ou mutando exemplos, roda o sistema e observa. Entradas que exploram partes novas viram base para as próximas.

	Software clássico	**Machine learning**
Entrada	bytes, arquivos, texto	imagens e textos levemente alterados
Falha	crash ou exceção	predição muda sem motivo: a foto quase igual vira outra classe
Guia da busca	cobertura de código: linhas executadas	cobertura de neurônios: neurônios ativados

MODELOS USADOS

Dois classificadores de imagem e um tokenizer de texto

ResNet-50

Para que serve: reconhecer o objeto de uma foto (cachorro, carro, xícara) entre 1000 classes.

Como: rede convolucional de 50 camadas; filtros detectam bordas, texturas e formas.

ViT-Base

Para que serve: a mesma tarefa da ResNet, com as mesmas 1000 classes.

Como: corta a imagem em quadrados de 16×16 pixels e usa atenção, a técnica dos modelos de linguagem.

Tokenizer DistilBERT

Para que serve: quebrar texto em pedaços (tokens) e trocar cada um por um número que o modelo entende.

Por que testar: todo texto passa por ele; um erro aqui afeta tudo depois.

COMO GERAMOS OS TESTES

Mutar, verificar, guardar o que explora mais

Imagens (ResNet, ViT)

1. Muta uma imagem: ruído, brilho, contraste, rotação, blur ou patch
2. Limita a mudança a 0,08 por pixel
3. **Falha** se a classe prevista mudar
4. Guarda o mutante se ativar neurônios novos

Tokenizer (Atheris)

1. Gera texto Unicode e um max_length
2. Guarda entradas que executam código novo
3. **Falha** se: lançar exceção, passar do max_length, gerar id inválido ou decode/encode não estabilizar

⚠ Sem acesso ao Hugging Face: modelos com **pesos aleatórios**, imagens sintéticas e vocabulário local.

RESULTADOS · IMAGENS

ViT: 45 falhas em 500. ResNet: nenhuma.

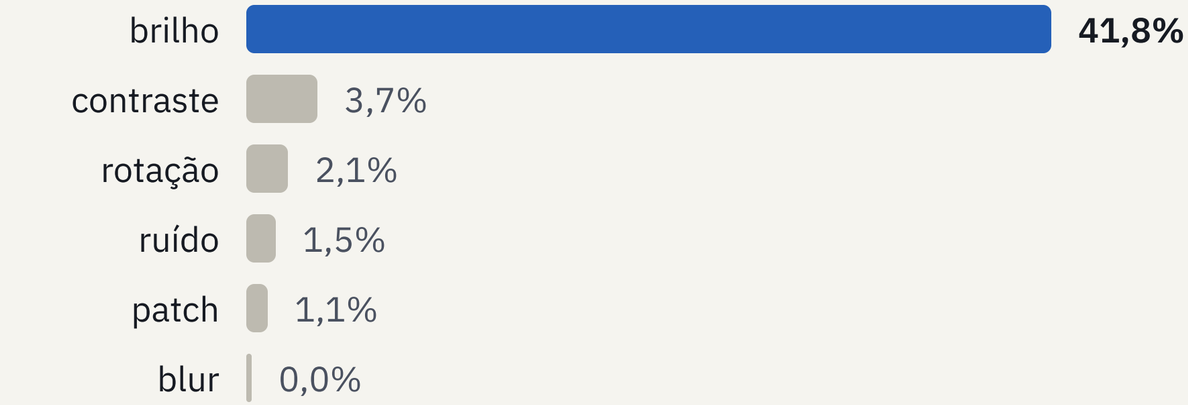
ViT-Base

45 / 500

ResNet-50

0 / 500

No ViT, o brilho causou 38 das 45 falhas



RESULTADOS · TOKENIZER

O tokenizer em Python estourou o max_length

Python puro · max_length=2 · texto 'Ƒ'

`[CLS] x [SEP]`

3 tokens: falha, achada em menos de 1 s

Rápido (Rust) · mesma entrada

`[CLS] [SEP]`

2 tokens: correto

O tokenizer rápido rodou 259 mil vezes sem falhas, mas o Atheris não enxerga código Rust: na prática, foi fuzzing aleatório.

CONCLUSÃO

O pipeline funciona; os números ainda não medem os modelos reais

- Com pesos aleatórios, a ResNet sempre responde a mesma classe e o ViT está quase indeciso
- Cobertura de neurônios não ajudou a achar falhas
- No tokenizer, o fuzzing só funciona onde o Atheris enxerga o código

Próximo passo: rodar com os pesos reais do Hugging Face e fotos de verdade.